

TP — Conteneurisation & Intégration Continue

Rendu : une URL de dépôt GitHub **public** + une URL d'image sur Docker Hub

1. Choisissez votre techno

Le TD réalisé en cours utilisait **Node.js / Express**. Pour ce TP, vous devez utiliser une **autre** technologie. Trois sont proposées — **choisissez-en une** :

Techno	Annexe à suivre	Profil
Java	ANNEXE-JAVA.md	langage compilé, image de build lourde → le multi-stage est spectaculaire
C#	ANNEXE-DOTNET.md	langage compilé, outillage Docker officiel très propre
PHP	ANNEXE-PHP.md	langage interprété : le multi-stage porte sur Composer et les extensions natives, et l'API tourne en php-fpm derrière nginx

Les trois parcours sont notés avec **le même barème** et exigent **le même travail**. Le sujet ci-dessous est commun ; l'annexe de votre techno vous donne le code de l'application (fourni, à recopier tel quel) et les quelques valeurs qui diffèrent — port, images de base, endpoint de santé, variables d'environnement.

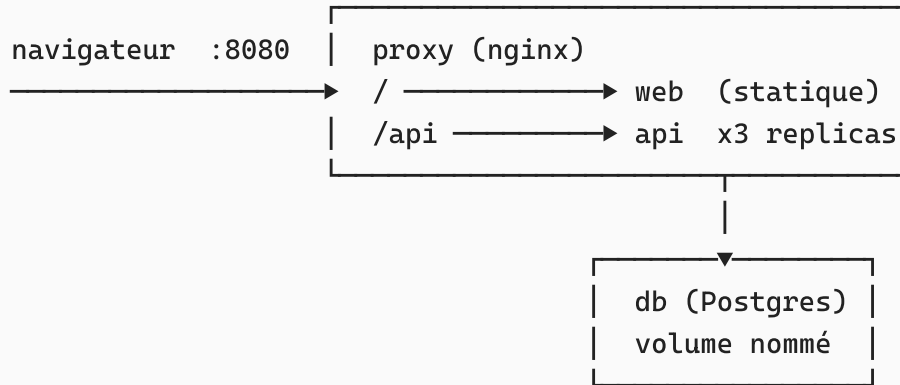
Annoncez votre choix dans le `README.md` **de votre dépôt.**

Le TP note **Docker et la CI**, pas votre maîtrise du langage. Le code applicatif est fourni intégralement dans l'annexe : vous n'avez pas à l'écrire ni à le modifier.

2. Contexte et architecture cible

Vous devez livrer une application complète qui se lance avec **une seule commande** :

```
docker compose up -d
```



Trois réseaux cloisonnés (comme dans le TD) :

Réseau	Services connectés	Rôle
proxy-front	proxy, web	le proxy sert le front
proxy-back	proxy, api	le proxy joint l'API
backend	api, db	l'API joint la base

Contrainte forte : le service `web` ne doit **pas** pouvoir joindre `db`, et `db` ne doit exposer **aucun port** sur la machine hôte. Vous devrez le prouver (§8).

Contrat d'API — identique pour les trois technos

Méthode	Route	Rôle
GET	/api/taches	liste les tâches (JSON)
POST	/api/taches	crée une tâche — corps : <code>{"titre":"...", "faite":false}</code>
GET	/api/qui	renvoie le <i>hostname</i> du conteneur — sert à vérifier le load-balancing

Arborescence attendue

```

tp-docker-ci/
├─ compose.yml
├─ nginx-proxy.conf
├─ .env.example           (commité) / .env (ignoré)
├─ db/
│   └─ init.sql
├─ api/
│   └─ Dockerfile
  
```

```

|   ├── .dockerignore
|   └── ...
├── web/
|   ├── Dockerfile
|   └── src/index.html
├── .github/workflows/ci.yml
├── README.md
└── RAPPORT.md

```

← contenu fourni par votre annexe

Fichiers communs aux trois technos

db/init.sql

```

CREATE TABLE tache (
  id      BIGSERIAL PRIMARY KEY,
  titre   VARCHAR(255) NOT NULL,
  faite   BOOLEAN NOT NULL DEFAULT FALSE
);

INSERT INTO tache (titre, faite) VALUES
  ('Écrire le Dockerfile multi-stage', false),
  ('Cloisonner les réseaux', false),
  ('Faire passer la CI au vert', false);

```

web/Dockerfile

```

FROM nginx:alpine
COPY ./src /usr/share/nginx/html

```

web/src/index.html

```

<!doctype html>
<html lang="fr">
<head><meta charset="utf-8"><title>TP Docker</title></head>
<body>
  <h1>Mes tâches</h1>
  <ul id="liste"></ul>
  <p>Servi par : <span id="qui">...</span></p>
  <script>
    fetch('/api/taches').then(r => r.json()).then(taches => {
      document.getElementById('liste').innerHTML =
        taches.map(t => `<li>${t.fait ? '☑' : '☐'} ${t.titre}</li>`).join('');
    });
  </script>

```

```

    fetch('/api/qui').then(r => r.text())
      .then(h => document.getElementById('qui').textContent = h);
  </script>
</body>
</html>

```

3. Tableau de correspondance des technos

Toutes les valeurs qui changent d'une techno à l'autre sont regroupées ici. Votre annexe les détaille.

	Java / Spring Boot	C# / ASP.NET Core	PHP / Slimphp-fpm
Image de build	maven:3.9-eclipse-temurin-21-alpine	mcr.microsoft.com/dotnet/sdk:10.0-alpine	composer
Image finale	eclipse-temurin:21-jre-alpine	mcr.microsoft.com/dotnet/aspnet:10.0-alpine	php:8.4-alpine
Ce que le multi-stage élimine	JDK + Maven + ~/.m2 + sources	SDK + NuGet + sources	Compose têtes de compilation extension
Manifeste de dépendances (à copier en premier)	pom.xml	*.csproj	composer + composer
Port du conteneur	8080 (HTTP)	8080 (HTTP)	9000 (FastCGI HTTP)
Le proxy y accède par	proxy_pass	proxy_pass	fastcgi_
Sonde de santé	GET /actuator/health	GET /health	cgi-fcgi /ping
Où lire la version pour la CI	<version> du pom.xml	<Version> du .csproj	version composer

4. Partie 1 — Le Dockerfile de l'API (6 pts)

C'est la partie la plus importante. Écrivez `api/Dockerfile` en respectant **toutes** les exigences ci-dessous.

Exigences

1. Build multi-stage obligatoire.

- Étape 1 (`build`) : l'image d'outillage de votre techno (voir §3), qui produit l'artefact déployable (un `.jar`, des `.dll`, ou un dossier `vendor/`).
- Étape 2 (finale) : l'image d'exécution minimale, dans laquelle vous copiez **uniquement** l'artefact depuis l'étape de build, avec `COPY --from=...`.
- L'image finale ne doit contenir **aucun outil de build** (ni compilateur, ni gestionnaire de paquets, ni sources non nécessaires à l'exécution).

2. Optimisation du cache de layers.

Le téléchargement des dépendances prend de 30 s à 2 min. Organisez les instructions pour qu'une modification du **code source** ne relance **pas** ce téléchargement.

*Indice : c'est exactement le raisonnement de ``COPY package.json ./ **avant** COPY ./index.js`` dans le TD Node. Chaque techno a son équivalent (§3).**

3. Sécurité.

- Images de base **pinnées par digest** (`image:tag@sha256:...`), pas seulement par tag. Récupérez-les avec `docker buildx imagetools inspect <image>`.
- Le conteneur ne tourne **pas en root** : utilisez `USER` (voir votre annexe pour l'utilisateur adapté).
- Les fichiers de l'application appartiennent à cet utilisateur.

4. HEALTHCHECK fonctionnel : l'état du conteneur doit passer à `healthy` dans `docker compose ps` (voir la sonde de votre techno en §3).

5. `.dockerignore` . Il doit exclure au minimum le dossier de build local, `.git`, `.github`, `*.md` et `.env` . **Expliquez en commentaire** pourquoi exclure le dossier de build local est critique.

Vérifications à consigner dans le rapport

```
docker image build -t tp-api:0.1.0 ./api
docker image ls tp-api                                # taille : objectif §3

docker image inspect tp-api:0.1.0 --format '{{.Config.User}}' # pas root,
pas vide

# Absence de l'outillage de build dans l'image finale
# Java : which mvn
```

```
# C# : which dotnet (doit être le runtime, pas le SDK : dotnet --list-sdks
vide)
# PHP : which composer
docker container run --rm --entrypoint sh tp-api:0.1.0 -c "which <outil> ||
echo 'absent : OK'"

# Le cache fonctionne : modifiez un fichier source, relancez le build,
# et vérifiez que l'étape de récupération des dépendances affiche CACHED
docker image build -t tp-api:0.1.0 ./api
```

5. Partie 2 — `compose.yml` (5 pts)

Écrivez `compose.yml` à la racine, avec les 4 services `proxy`, `web`, `api`, `db`.

1. **Réseaux** : les trois réseaux du §2, avec exactement les rattachements indiqués.
2. **Volume nommé** pour les données Postgres : une tâche ajoutée via `POST /api/taches` doit survivre à un `docker compose down` **sans** `-v`.
3. **api en 3 replicas**, avec une limite de ressources (`cpus` **et** mémoire).
`api` ne publie **aucun port** sur l'hôte.
4. **Aucun secret en clair** dans `compose.yml` : les identifiants de la base viennent d'un fichier `.env` non commité, avec un `.env.example` commité à côté.
5. Seul `proxy` publie un port : `8080:80`.

6. Partie 3 — Reverse proxy nginx (3 pts)

Écrivez `nginx-proxy.conf`, monté dans le service `proxy`.

1. `location /` → service `web`.
2. `location /api` → les replicas de `api`, via un bloc `upstream` (`proxy_pass` ou `fastcgi_pass` selon votre techno, §3).
3. **Load-balancing** avec la stratégie `least_conn`.
4. **Rate limiting** sur `/api` : 10 req/s par IP, avec un `burst` de 20.
5. Transmission de l'IP réelle du client (`X-Real-IP`, `X-Forwarded-For`).
(Parcours PHP : via les `fastcgi_param` correspondants.)

Vérifications

```
# Load-balancing : les 3 hostnames doivent apparaître
for i in $(seq 1 12); do curl -s http://localhost:8080/api/qui; echo; done

# Rate limiting : on doit voir apparaître des 503
for i in $(seq 1 60); do curl -s -o /dev/null -w "%{http_code} "
http://localhost:8080/api/taches; done
```

7. Partie 4 — Intégration continue (5 pts)

Écrivez `.github/workflows/ci.yml`. Déclenchement sur `push` **et** `pull_request` vers `main`.

1. **Job** `test` : installe l'outillage de votre techno (`actions/setup-java` , `actions/setup-dotnet` ou `shivammathur/setup-php`) **avec le cache de dépendances activé**, puis lance la commande de test/validation de votre annexe. Ce job tourne sur **toutes** les PR.
2. **Job** `docker` : dépend de `test` (`needs:`) et ne s'exécute **que sur** `main` — le push d'image ne doit jamais partir d'une branche de contribution.
3. Le job `docker` doit :
 - se connecter à Docker Hub avec un **token** dans `secrets` et le login dans `vars` (aucun secret en clair dans le YAML) ;
 - **extraire automatiquement la version depuis le manifeste** du projet (§3) — aucune version recopiée à la main dans le workflow ;
 - builder et pousser l'image avec **deux tags** : `<version>` et `latest` .
4. L'image est publiée sur `docker.io/<votre-login>/tp-msii-api` .

Vérifications

- Ouvrez une PR contenant une erreur volontaire → le job `test` doit être **rouge** et **aucune image** ne doit être poussée.
- Corrigez, mergez sur `main` → les deux jobs passent au vert, l'image apparaît sur Docker Hub avec ses deux tags.
- Fournissez les **captures des deux exécutions** et l'URL Docker Hub.

8. Partie 5 — Vérification du cloisonnement réseau (1 pt)

Prouvez, commandes et sorties à l'appui dans le rapport, que :

1. Depuis un conteneur `web` , il est **impossible** de joindre `db` ;
2. Depuis un conteneur `api` , il **est** possible de joindre `db` ;
3. Le port `5432` n'est **pas** ouvert sur la machine hôte.

Rappel des commandes utiles

```
docker compose up -d --build      # démarre tout
docker compose ps                 # état + healthcheck
docker compose logs -f api        # logs d'un service
docker compose exec api sh        # shell dans un conteneur
docker compose down               # arrête (garde le volume)
docker compose down -v            # arrête ET supprime le volume
docker compose up -d --scale api=5

docker image history tp-api:0.1.0  # layers et leur poids
docker buildx imagetools inspect <image> # récupérer un digest
docker system df                  # espace disque occupé
```

Bon courage.